

# Detailed Solution: Support Vector Machines (SVM)

## Theory Questions

### Concise Theory Summary (for a 5-mark question):

#### 1. Support Vector Machines (SVM) Fundamentals:

- **Objective:** To find an optimal hyperplane that best separates data points into different classes, maximizing the margin (distance to the nearest data points, called support vectors).
- **Hard Margin:** Assumes linearly separable data, strict separation. Sensitive to outliers.
- **Soft Margin:** Allows some misclassifications or margin violations using a regularization parameter  $C$ . Small  $C$  allows more errors for a wider margin (less overfitting), large  $C$  minimizes errors for a narrower margin (more prone to overfitting).

#### 2. The Kernel Trick:

- **Purpose:** To handle non-linearly separable data by implicitly mapping it into a higher-dimensional feature space where it becomes linearly separable.
- **Mechanism:** Uses kernel functions (e.g., Linear, Polynomial, RBF) to calculate dot products in the higher-dimensional space without explicit transformation, reducing computational cost.

#### 3. Multi-class Classification:

- SVMs are inherently binary. For multiple classes, strategies like:
  - **One-vs-Rest (OvR):** Trains  $N$  classifiers, each separating one class from all others. Predicts the class with the highest score.
  - **One-vs-One (OvO):** Trains  $N * (N-1) / 2$  classifiers, each separating two classes. Predicts based on a voting scheme.

## Theory Summary (Key Points):

#### 1. Role of Margin in SVM and Soft-Margin Classifier:

- **Margin:** The separation between the decision boundary (hyperplane) and the closest data points (support vectors) of each class. Maximizing this margin is a primary goal for better generalization.
- **Hard-Margin SVM:** Requires perfect linear separation; susceptible to noise and outliers.
- **Soft-Margin SVM:** Allows some misclassifications or margin violations to handle non-linearly separable data and outliers.
- **$C$  Parameter:** Controls the trade-off:
  - **Small  $C$ :** Prioritizes larger margin, more misclassifications allowed (more tolerant to outliers, less prone to overfitting).
  - **Large  $C$ :** Prioritizes fewer misclassifications, smaller margin (less tolerant to outliers, prone to overfitting).

## 2. The Kernel Trick and Common Kernels:

- **Kernel Trick:** A technique to handle non-linearly separable data by implicitly mapping it into a higher-dimensional feature space where it becomes linearly separable, without explicitly computing the coordinates in that space.
- **Kernel Function:** Calculates the inner product of data points as if they were in the higher-dimensional space,  $K(x_i, x_j) = \phi(x_i) \cdot \phi(x_j)$ .
- **Common Kernels:**
  - **Linear Kernel:**  $K(x_i, x_j) = x_i \cdot x_j$ . For linearly separable data.
  - **Polynomial Kernel:**  $K(x_i, x_j) = (\gamma * x_i \cdot x_j + r)^d$ . For non-linear, polynomial decision boundaries.
  - **RBF (Gaussian) Kernel:**  $K(x_i, x_j) = \exp(-\gamma * ||x_i - x_j||^2)$ . Very flexible, maps to infinite dimensions, good for complex non-linear relationships.

## 3. ROC Curve and AUC:

- **ROC Curve (Receiver Operating Characteristic):** A plot of True Positive Rate (TPR/Sensitivity) vs. False Positive Rate (FPR/1-Specificity) at various classification thresholds.
  - Ideal curve is top-left (TPR=1, FPR=0).
  - Random classifier is the diagonal line (AUC=0.5).
- **AUC (Area Under the Curve):** A single scalar metric summarizing the overall performance of a binary classifier across all possible thresholds.
  - Ranges from 0 to 1.
  - Interpreted as the probability that the model ranks a random positive instance higher than a random negative instance.

## 4. How SVM handles More Than Two Classes:

- SVMs are inherently binary classifiers.
- **One-vs-Rest (OvR) / One-vs-All (OvA):**
  - Trains  $N$  binary SVMs, where each SVM distinguishes one class from all other  $N - 1$  classes combined.
  - Prediction: chooses the class with the highest decision score.
  - Pros: Simple. Cons: Can suffer from imbalanced datasets, ambiguous regions.
- **One-vs-One (OvO):**
  - Trains  $N * (N - 1) / 2$  binary SVMs, each distinguishing between a specific pair of classes.
  - Prediction: uses a voting scheme; the class with most votes wins.
  - Pros: Each SVM trains on smaller data, better for imbalanced classes. Cons: Many classifiers, computationally expensive for large  $N$ .

## 1. What is the role of the margin in SVM? How does a soft-margin classifier handle outliers?

**Role of the Margin:** In Support Vector Machines (SVMs), the 'margin' refers to the separation between the decision boundary (hyperplane) and the closest data points from each class. These closest points are called **support vectors**. The primary goal of a hard-margin SVM is to find a

hyperplane that maximizes this margin. A larger margin generally leads to better generalization performance, meaning the model is more likely to correctly classify unseen data.

**Soft-Margin Classifier and Outliers:** A hard-margin SVM requires that all training examples are perfectly separated into their respective classes, with no data points falling into the margin or on the wrong side of the hyperplane. This works well for linearly separable data without noise or outliers. However, in real-world scenarios, data often contains outliers or is not perfectly linearly separable.

This is where a **soft-margin SVM** comes into play. A soft-margin classifier allows for some misclassifications or violations of the margin. It introduces a regularization parameter, typically denoted as  $C$  (or sometimes  $\lambda$ ), and slack variables ( $\xi - x_i$ ) for each data point. The objective of a soft-margin SVM is to:

1. Maximize the margin.
2. Minimize the number of margin violations and misclassifications.

The  $C$  parameter controls the trade-off between these two objectives:

- **Small  $C$ :** Prioritizes a larger margin, allowing more misclassifications. This makes the model more tolerant to outliers and can prevent overfitting, but might lead to higher training error.
- **Large  $C$ :** Prioritizes correct classification of training data, leading to a smaller margin and fewer misclassifications. This makes the model less tolerant to outliers and can lead to overfitting if the data is noisy.

By allowing some errors, the soft-margin classifier becomes more robust to outliers and works effectively with non-linearly separable data, finding a reasonable balance between maximizing the margin and minimizing classification errors.

## 2. Explain the kernel trick. Name three common kernels.

**The Kernel Trick:** The kernel trick is a powerful technique used in SVMs (and other algorithms) to handle non-linearly separable data. When data is not linearly separable in its original feature space, a standard linear SVM cannot find a straight hyperplane to perfectly separate the classes. The kernel trick addresses this by implicitly mapping the data into a higher-dimensional feature space where it *might* become linearly separable, without explicitly calculating the coordinates of the data in that higher-dimensional space.

Instead of transforming the data points themselves, the kernel trick applies a **kernel function** ( $K$ ) that calculates the inner product between two data points *as if* they had been mapped to the higher-dimensional space. If  $\phi(x)$  is the mapping function that transforms  $x$  into the higher-dimensional space, then the kernel function  $K(x_i, x_j)$  computes  $\phi(x_i) \cdot \phi(x_j)$  directly. This avoids the computational expense of explicitly computing  $\phi(x)$  for every data point and then performing the dot product, which can be extremely high or even infinite-dimensional.

Essentially, the kernel trick allows a linear classifier to learn a non-linear decision boundary in the original feature space by effectively operating in a higher-dimensional space.

### Three Common Kernels:

1. **Linear Kernel:**  $K(x_i, x_j) = x_i \cdot x_j$ 
  - This is the simplest kernel. It's equivalent to performing a linear SVM in the original feature space. It's suitable for linearly separable data and acts as a baseline.
2. **Polynomial Kernel:**  $K(x_i, x_j) = (\gamma * x_i \cdot x_j + r)^d$ 
  - $d$  is the degree of the polynomial,  $\gamma$  (gamma) is a scaling factor, and  $r$  (coef0) is a constant term. This kernel allows for non-linear decision boundaries that are polynomial in nature. Higher degrees can fit more complex relationships but are prone to overfitting.
3. **Radial Basis Function (RBF) Kernel (or Gaussian Kernel):**  $K(x_i, x_j) = \exp(-\gamma * ||x_i - x_j||^2)$ 
  - $\gamma$  (gamma) is a parameter that defines how much influence a single training example has. This is one of the most widely used kernels. It can map data into an infinite-dimensional space and is very flexible, capable of handling complex non-linear relationships. It generally performs well across various datasets but requires careful tuning of  $\gamma$  and  $C$ .

3. What does an ROC curve plot? What does AUC represent?

**ROC Curve (Receiver Operating Characteristic Curve):** An ROC curve is a graphical plot that illustrates the diagnostic ability of a binary classifier system as its discrimination threshold is varied. It plots two parameters:

- **True Positive Rate (TPR) / Sensitivity / Recall** on the y-axis:  $TPR = TP / (TP + FN)$  (proportion of actual positives correctly identified)
- **False Positive Rate (FPR) / (1 - Specificity)** on the x-axis:  $FPR = FP / (FP + TN)$  (proportion of actual negatives incorrectly identified as positive)

The curve is generated by varying the classification threshold for the predicted probabilities or decision scores. Each point on the ROC curve represents a sensitivity/specificity pair corresponding to a particular decision threshold.

- A perfect classifier would have a point at (0, 1), meaning it has 0% FPR and 100% TPR.
- A completely random classifier would trace a diagonal line from (0, 0) to (1, 1).

**AUC (Area Under the ROC Curve):** AUC represents the **Area Under the ROC Curve**. It is a single scalar value that summarizes the overall performance of a binary classification model across all possible classification thresholds. AUC provides an aggregate measure of performance, independent of the chosen threshold.

#### Interpretation of AUC:

- **AUC = 1:** A perfect classifier (discriminates between all positive and negative class points correctly).
- **AUC = 0.5:** A classifier performing no better than random guessing.
- **AUC < 0.5:** A classifier that performs worse than random guessing (e.g., predicting positives as negatives and vice-versa). In such cases, one could simply invert the predictions to get an  $AUC > 0.5$ .

More generally, the AUC can be interpreted as the probability that the classifier will rank a randomly chosen positive instance higher than a randomly chosen negative instance. A higher AUC indicates a better model performance in distinguishing between the positive and negative classes.

#### 4. How does SVM handle more than two classes?

Support Vector Machines are inherently binary classifiers, meaning they are designed to distinguish between only two classes. To extend SVMs to handle multi-class classification problems (where there are more than two classes), two main strategies are commonly used:

##### 1. **One-vs-Rest (OvR) or One-vs-All (OvA):**

- **Concept:** This strategy involves training  $N$  separate binary SVM classifiers, where  $N$  is the number of classes. For each classifier, one class is treated as the 'positive' class, and all the other  $N - 1$  classes are grouped together and treated as the 'negative' class.
- **Training:** If you have classes A, B, and C, you would train:
  - Classifier 1: Distinguishes A vs. (B and C)
  - Classifier 2: Distinguishes B vs. (A and C)
  - Classifier 3: Distinguishes C vs. (A and B)
- **Prediction:** To classify a new data point, each of the  $N$  classifiers generates a decision score (e.g., distance from the hyperplane). The class whose classifier yields the highest decision score is chosen as the predicted class.
- **Advantages:** Relatively straightforward to implement, and each SVM only needs to distinguish one class from the rest.
- **Disadvantages:** Can suffer from imbalanced datasets during training (the 'rest' class is usually much larger). Also, ambiguous regions can exist where multiple classifiers predict a positive class, or none do.

##### 2. **One-vs-One (OvO):**

- **Concept:** This strategy involves training a binary SVM classifier for every possible pair of classes. If there are  $N$  classes, this will result in  $N * (N - 1) / 2$  classifiers.
- **Training:** For classes A, B, and C, you would train:
  - Classifier 1: Distinguishes A vs. B
  - Classifier 2: Distinguishes A vs. C
  - Classifier 3: Distinguishes B vs. C
- **Prediction:** To classify a new data point, each of the  $N * (N - 1) / 2$  classifiers is run, and the class that receives the most 'votes' (i.e., is predicted as positive most frequently across all pairwise comparisons) is selected as the final prediction.
- **Advantages:** Each SVM is trained on a smaller subset of the data, which can be faster for certain problems. It naturally handles imbalanced classes better as each classifier only sees two classes.
- **Disadvantages:** Requires training a much larger number of classifiers, which can be computationally expensive if  $N$  is large. Vote counting can sometimes lead to ties.

## Coding Exercises

### Exercise 1: Binary SVM with Linear and RBF Kernels

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Generate and split data
X, y = make_classification(n_samples=300, n_features=2, n_redundant=0,
                          n_clusters_per_class=1, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)

print("Data generated and split successfully.")
print(f"X_train shape: {X_train.shape}, y_train shape: {y_train.shape}")
print(f"X_test shape: {X_test.shape}, y_test shape: {y_test.shape}")

Data generated and split successfully.
X_train shape: (210, 2), y_train shape: (210,)
X_test shape: (90, 2), y_test shape: (90,)
```

### Explanation of Code Cell 6d8727a5:

This cell initializes the environment for the first coding exercise by importing necessary libraries and generating a synthetic dataset.

```
import numpy as np # Import NumPy for numerical operations, often used
with arrays.
import matplotlib.pyplot as plt # Import Matplotlib for plotting and
visualization.
from sklearn.datasets import make_classification # Import a function
to generate synthetic classification datasets.
from sklearn.svm import SVC # Import the Support Vector Classifier
model.
from sklearn.model_selection import train_test_split # Import a
function to split data into training and testing sets.
from sklearn.metrics import accuracy_score # Import a metric to
evaluate model performance (accuracy).

# Generate and split data
X, y = make_classification(n_samples=300, n_features=2, n_redundant=0,
# Generate 300 samples, 2 features, no redundant features,
                          n_clusters_per_class=1, random_state=42) #
1 cluster per class, for reproducibility.
X_train, X_test, y_train, y_test = train_test_split(X, y,
```

```

test_size=0.3, random_state=42) # Split data: 70% train, 30% test, for
reproducibility.

print("Data generated and split successfully.") # Confirm data
generation.
print(f"X_train shape: {X_train.shape}, y_train shape:
{y_train.shape}") # Print shapes of training data.
print(f"X_test shape: {X_test.shape}, y_test shape: {y_test.shape}") #
Print shapes of testing data.

```

Task 1: Train two SVM models (linear and RBF) and report test accuracy.

```

# 1. Train two SVM models: linear kernel and RBF kernel (C=1.0).
Report test accuracy.

```

```

# Linear Kernel SVM

```

```

svm_linear = SVC(kernel='linear', C=1.0, random_state=42)
svm_linear.fit(X_train, y_train)
y_pred_linear = svm_linear.predict(X_test)
accuracy_linear = accuracy_score(y_test, y_pred_linear)
print(f"Test Accuracy (Linear Kernel): {accuracy_linear:.4f}")

```

```

# RBF Kernel SVM (C=1.0)

```

```

svm_rbf = SVC(kernel='rbf', C=1.0, random_state=42)
svm_rbf.fit(X_train, y_train)
y_pred_rbf = svm_rbf.predict(X_test)
accuracy_rbf = accuracy_score(y_test, y_pred_rbf)
print(f"Test Accuracy (RBF Kernel, C=1.0): {accuracy_rbf:.4f}")

```

```

Test Accuracy (Linear Kernel): 0.9556
Test Accuracy (RBF Kernel, C=1.0): 0.9667

```

## Explanation of Code Cell 25b9b4c0:

This cell trains two Support Vector Machine (SVM) models with different kernels (linear and RBF) and evaluates their performance using test accuracy.

```

# 1. Train two SVM models: linear kernel and RBF kernel (C=1.0).
Report test accuracy.

```

```

# Linear Kernel SVM

```

```

svm_linear = SVC(kernel='linear', C=1.0, random_state=42) # Initialize
an SVC model with a linear kernel, C=1.0, and a random state for
reproducibility.
svm_linear.fit(X_train, y_train) # Train the linear SVM model using
the training data.
y_pred_linear = svm_linear.predict(X_test) # Make predictions on the
test data.
accuracy_linear = accuracy_score(y_test, y_pred_linear) # Calculate
the accuracy of the linear SVM.

```

```

print(f"Test Accuracy (Linear Kernel): {accuracy_linear:.4f}") # Print
the test accuracy.

# RBF Kernel SVM (C=1.0)
svm_rbf = SVC(kernel='rbf', C=1.0, random_state=42) # Initialize an
SVC model with an RBF kernel, C=1.0, and a random state.
svm_rbf.fit(X_train, y_train) # Train the RBF SVM model using the
training data.
y_pred_rbf = svm_rbf.predict(X_test) # Make predictions on the test
data.
accuracy_rbf = accuracy_score(y_test, y_pred_rbf) # Calculate the
accuracy of the RBF SVM.
print(f"Test Accuracy (RBF Kernel, C=1.0): {accuracy_rbf:.4f}") #
Print the test accuracy.

```

Task 2: Plot decision boundaries for both models using contourf.

```

# Helper function to plot decision boundaries
def plot_decision_boundary(model, X, y, title):
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.02),
                          np.arange(y_min, y_max, 0.02))

    Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    plt.contourf(xx, yy, Z, alpha=0.8, cmap=plt.cm.coolwarm)
    plt.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.coolwarm, s=20,
edgecolors='k')
    plt.xlabel('Feature 1')
    plt.ylabel('Feature 2')
    plt.title(title)
    plt.show()

# 2. Plot decision boundaries for both models
plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)
plot_decision_boundary(svm_linear, X_train, y_train, 'Linear SVM
Decision Boundary')

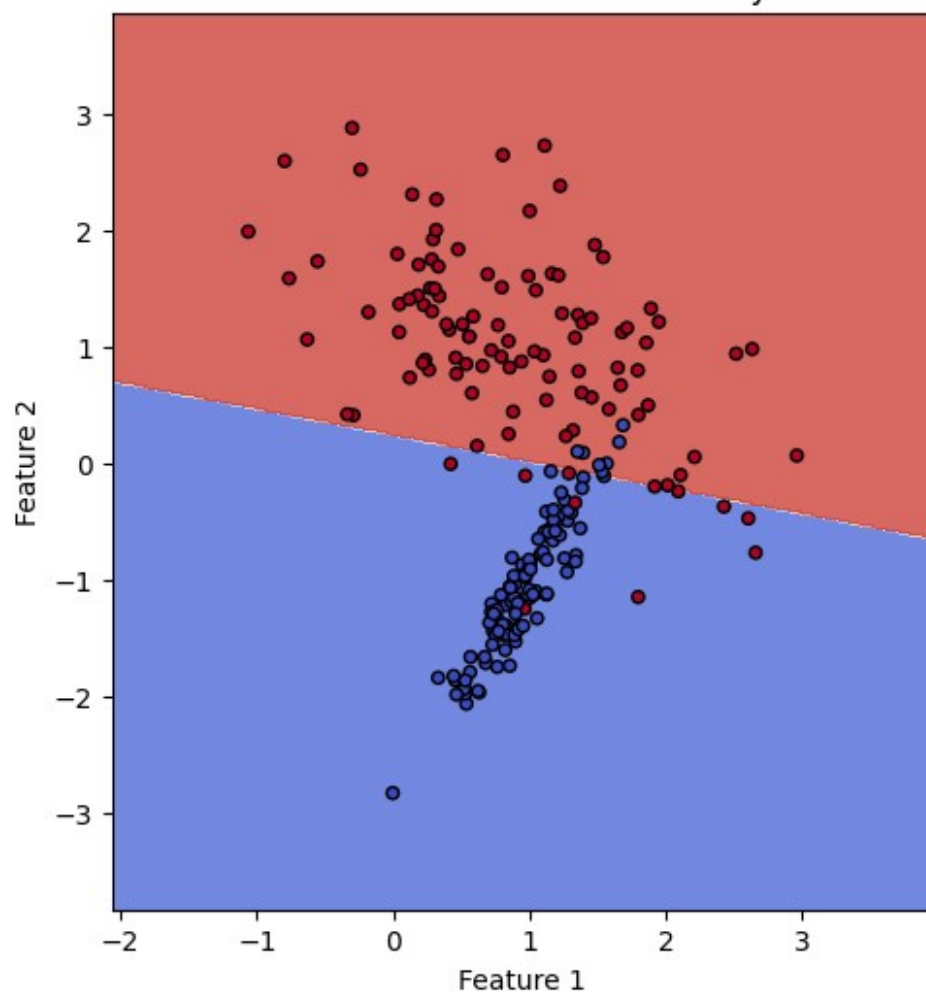
plt.subplot(1, 2, 2)
plot_decision_boundary(svm_rbf, X_train, y_train, 'RBF SVM (C=1.0)
Decision Boundary')

plt.tight_layout()
plt.show()

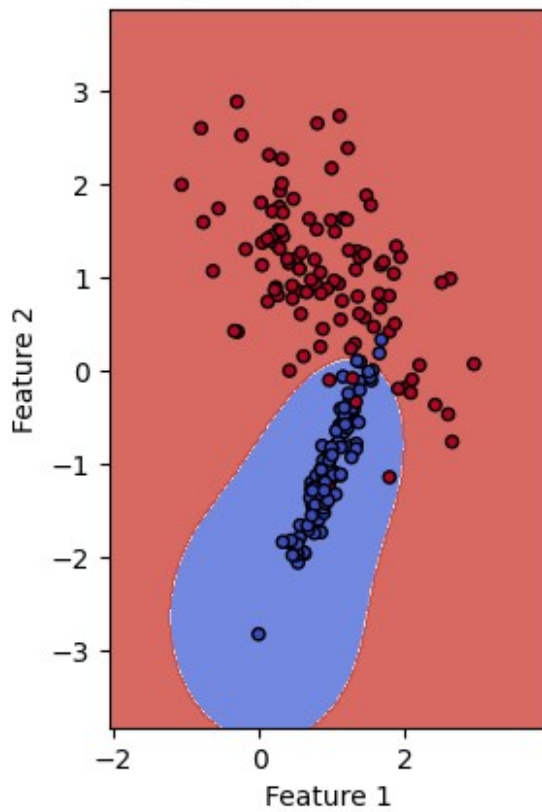
```



Linear SVM Decision Boundary



RBF SVM (C=1.0) Decision Boundary



<Figure size 640x480 with 0 Axes>

### Explanation of Code Cell 9d04aa9f:

This cell defines a helper function to plot the decision boundaries of SVM models and then uses it to visualize the boundaries for both the linear and RBF kernel SVMs trained previously.

```
# Helper function to plot decision boundaries
def plot_decision_boundary(model, X, y, title): # Define a function that takes a trained model, data (X, y), and a title.
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1 # Determine the range for the first feature (x-axis).
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1 # Determine the range for the second feature (y-axis).
    xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.02), # Create a meshgrid to cover the feature space with a step of 0.02.
                          np.arange(y_min, y_max, 0.02))

    Z = model.predict(np.c_[xx.ravel(), yy.ravel()]) # Predict the class for each point in the meshgrid.
    Z = Z.reshape(xx.shape) # Reshape the predictions back to the grid shape.
```

```

plt.contourf(xx, yy, Z, alpha=0.8, cmap=plt.cm.coolwarm) # Plot
the decision regions using contourf.
plt.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.coolwarm, s=20,
edgecolors='k') # Plot the actual data points, colored by class.
plt.xlabel('Feature 1') # Label the x-axis.
plt.ylabel('Feature 2') # Label the y-axis.
plt.title(title) # Set the plot title.
plt.show() # Display the plot.

# 2. Plot decision boundaries for both models
plt.figure(figsize=(12, 6)) # Create a figure with a specified size
for two subplots.

plt.subplot(1, 2, 1) # Create the first subplot (1 row, 2 columns,
first plot).
plot_decision_boundary(svm_linear, X_train, y_train, 'Linear SVM
Decision Boundary') # Plot the decision boundary for the linear SVM.

plt.subplot(1, 2, 2) # Create the second subplot.
plot_decision_boundary(svm_rbf, X_train, y_train, 'RBF SVM (C=1.0)
Decision Boundary') # Plot the decision boundary for the RBF SVM.

plt.tight_layout() # Adjust subplot parameters for a tight layout.
plt.show() # Display both plots.

```

Task 3: Experiment with  $C = 0.01, 1, 100$  for RBF kernel. How does decision boundary change?

```

# 3. Experiment with C = 0.01, 1, 100 for RBF kernel
C_values = [0.01, 1, 100]

plt.figure(figsize=(15, 5))

for i, C in enumerate(C_values):
    svm_rbf_c = SVC(kernel='rbf', C=C, random_state=42)
    svm_rbf_c.fit(X_train, y_train)
    y_pred_rbf_c = svm_rbf_c.predict(X_test)
    accuracy_rbf_c = accuracy_score(y_test, y_pred_rbf_c)

    print(f"Test Accuracy (RBF Kernel, C={C}): {accuracy_rbf_c:.4f}")

    plt.subplot(1, len(C_values), i + 1)
    plot_decision_boundary(svm_rbf_c, X_train, y_train, f'RBF SVM
Decision Boundary (C={C})')

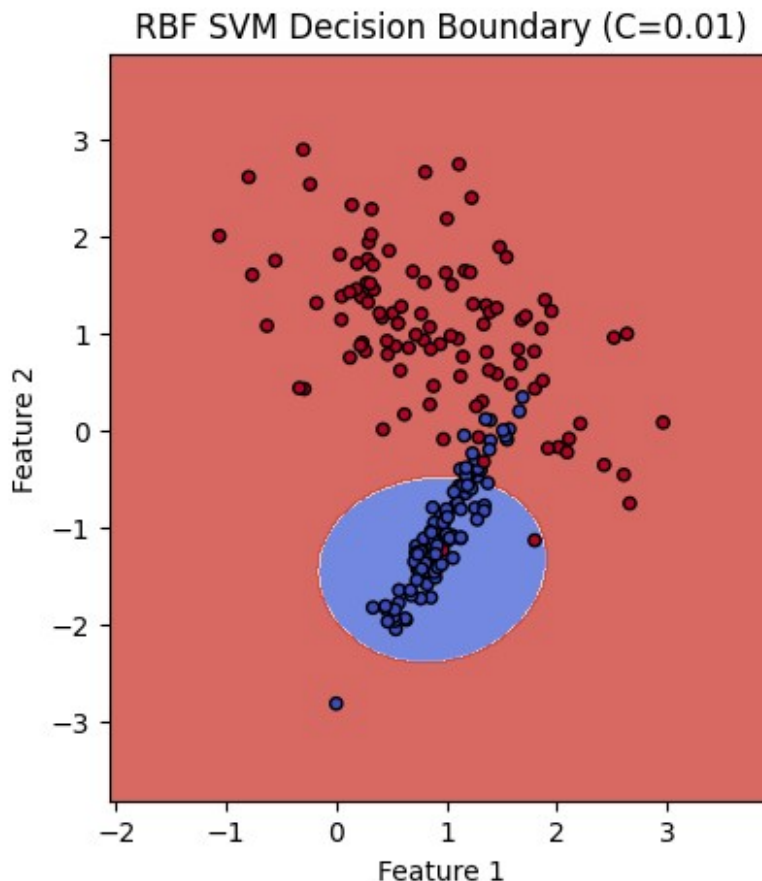
plt.tight_layout()
plt.show()

print("\nHow the decision boundary changes with C:")

```

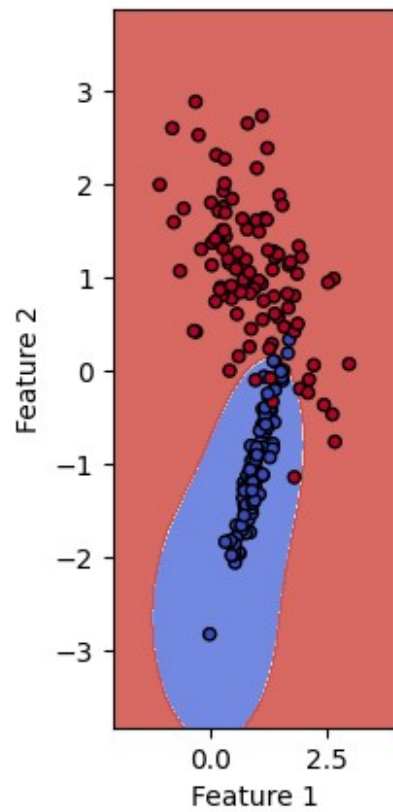
```
print("- **C = 0.01 (Small C):** The decision boundary is smoother and wider. The model prioritizes a larger margin, even if it means misclassifying some training points. It's more tolerant to misclassifications and might generalize better to unseen data if there's noise.")
print("- **C = 1 (Moderate C):** A good balance between margin maximization and error minimization. The boundary is less constrained than with C=0.01, allowing for slightly more complex shapes to better fit the training data, while still maintaining a reasonable margin.")
print("- **C = 100 (Large C):** The decision boundary becomes more complex and tightly fits the training data, attempting to correctly classify almost all training points. The margin becomes narrower. This can lead to overfitting, especially if the training data is noisy, as the model tries to accommodate every point, including potential outliers.")
```

Test Accuracy (RBF Kernel, C=0.01): 0.8889



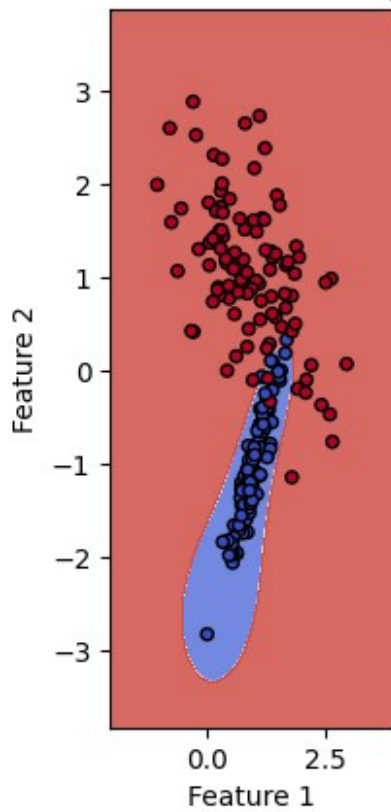
Test Accuracy (RBF Kernel, C=1): 0.9667

RBFSVM Decision Boundary (C=1)



Test Accuracy (RBF Kernel,  $C=100$ ): 0.9556

RBFSVM Decision Boundary (C=100)



<Figure size 640x480 with 0 Axes>

How the decision boundary changes with C:

- **C = 0.01 (Small C):** The decision boundary is smoother and wider. The model prioritizes a larger margin, even if it means misclassifying some training points. It's more tolerant to misclassifications and might generalize better to unseen data if there's noise.
- **C = 1 (Moderate C):** A good balance between margin maximization and error minimization. The boundary is less constrained than with  $C=0.01$ , allowing for slightly more complex shapes to better fit the training data, while still maintaining a reasonable margin.
- **C = 100 (Large C):** The decision boundary becomes more complex and tightly fits the training data, attempting to correctly classify almost all training points. The margin becomes narrower. This can lead to overfitting, especially if the training data is noisy, as the model tries to accommodate every point, including potential outliers.

## Explanation of Code Cell 4610470d:

This cell demonstrates the effect of the  $C$  parameter on the RBF kernel SVM's decision boundary and accuracy. It trains RBF SVMs with three different  $C$  values (0.01, 1, 100), calculates their test accuracy, and plots their respective decision boundaries.

```

# 3. Experiment with C = 0.01, 1, 100 for RBF kernel
C_values = [0.01, 1, 100] # Define a list of C values to experiment
with.

plt.figure(figsize=(15, 5)) # Create a figure with a specified size
for three subplots.

for i, C in enumerate(C_values): # Loop through each C value.
    svm_rbf_c = SVC(kernel='rbf', C=C, random_state=42) # Initialize
    an RBF SVM with the current C value.
    svm_rbf_c.fit(X_train, y_train) # Train the model.
    y_pred_rbf_c = svm_rbf_c.predict(X_test) # Make predictions.
    accuracy_rbf_c = accuracy_score(y_test, y_pred_rbf_c) # Calculate
    accuracy.

    print(f"Test Accuracy (RBF Kernel, C={C}): {accuracy_rbf_c:.4f}")
    # Print the accuracy for the current C.

    plt.subplot(1, len(C_values), i + 1) # Create a subplot for the
    current C value.
    plot_decision_boundary(svm_rbf_c, X_train, y_train, f'RBF SVM
    Decision Boundary (C={C})') # Plot the decision boundary.

plt.tight_layout() # Adjust subplot parameters for a tight layout.
plt.show() # Display all plots.

print("\nHow the decision boundary changes with C:") # Print
observations about the C parameter's effect.
print("- **C = 0.01 (Small C):** The decision boundary is smoother and
wider. The model prioritizes a larger margin, even if it means
misclassifying some training points. It's more tolerant to
misclassifications and might generalize better to unseen data if
there's noise.")
print("- **C = 1 (Moderate C):** A good balance between margin
maximization and error minimization. The boundary is less constrained
than with C=0.01, allowing for slightly more complex shapes to better
fit the training data, while still maintaining a reasonable margin.")
print("- **C = 100 (Large C):** The decision boundary becomes more
complex and tightly fits the training data, attempting to correctly
classify almost all training points. The margin becomes narrower. This
can lead to overfitting, especially if the training data is noisy, as
the model tries to accommodate every point, including potential
outliers.")

```

## Exercise 2: ROC and AUC

```

from sklearn.datasets import load_breast_cancer
from sklearn.metrics import roc_curve, auc, confusion_matrix,
roc_auc_score
from sklearn.preprocessing import StandardScaler

```

```

# Load Breast Cancer dataset
data = load_breast_cancer()
X, y = data.data, data.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)

# Scale features (important for SVM with RBF kernel)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Breast Cancer data loaded, split, and scaled successfully.")
Breast Cancer data loaded, split, and scaled successfully.

```

## Explanation of Code Cell 2160ad05:

This cell prepares the Breast Cancer dataset for the ROC and AUC exercise. It loads the data, splits it into training and testing sets, and scales the features, which is crucial for SVM performance.

```

from sklearn.datasets import load_breast_cancer # Import the Breast
Cancer dataset loader.
from sklearn.metrics import roc_curve, auc, confusion_matrix,
roc_auc_score # Import metrics for ROC analysis and confusion matrix.
from sklearn.preprocessing import StandardScaler # Import
StandardScaler for feature scaling.

# Load Breast Cancer dataset
data = load_breast_cancer() # Load the dataset.
X, y = data.data, data.target # Separate features (X) and target
labels (y).

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42) # Split data into 70% training and 30%
testing, ensuring reproducibility.

# Scale features (important for SVM with RBF kernel)
scaler = StandardScaler() # Initialize a StandardScaler.
X_train_scaled = scaler.fit_transform(X_train) # Fit the scaler on
training data and transform it.
X_test_scaled = scaler.transform(X_test) # Transform the test data
using the *same* scaler fitted on training data.

print("Breast Cancer data loaded, split, and scaled successfully.") #
Confirm successful data preparation.

```



Task 1: Train an RBF SVM. Compute predicted probabilities using `decision_function`.

```
# 1. Train an RBF SVM. Compute predicted probabilities using
decision_function.

# Train RBF SVM (C=1.0, default gamma='scale')
svm_rbf_bc = SVC(kernel='rbf', C=1.0, random_state=42,
probability=True) # probability=True is needed for predict_proba
svm_rbf_bc.fit(X_train_scaled, y_train)

# Get decision function scores
y_scores = svm_rbf_bc.decision_function(X_test_scaled)

# For ROC curve, we need either decision_function scores or
predict_proba.
# decision_function gives signed distance to hyperplane. Higher values
mean higher confidence in the positive class.
print(f"Shape of decision_function scores: {y_scores.shape}")
print(f"Sample decision_function scores: {y_scores[:5]}")

# We can also get probabilities if `probability=True` was set during
SVC initialization
y_proba = svm_rbf_bc.predict_proba(X_test_scaled)[: , 1]
print(f"Shape of predicted probabilities: {y_proba.shape}")
print(f"Sample predicted probabilities (positive class):
{y_proba[:5]}")

Shape of decision_function scores: (171,)
Sample decision_function scores: [ 1.09150282 -2.06431536 -1.62879474
 2.40170085  2.58841336]
Shape of predicted probabilities: (171,)
Sample predicted probabilities (positive class): [9.76322581e-01
 3.15264043e-04 1.60236621e-03 9.99999900e-01
 9.99999975e-01]
```

## Explanation of Code Cell 27a26398:

This cell trains an RBF kernel SVM on the scaled Breast Cancer data and then extracts the model's decision function scores and predicted probabilities. These values are essential for calculating and plotting the ROC curve.

```
# 1. Train an RBF SVM. Compute predicted probabilities using
decision_function.

# Train RBF SVM (C=1.0, default gamma='scale')
svm_rbf_bc = SVC(kernel='rbf', C=1.0, random_state=42,
probability=True) # Initialize an RBF SVM. `probability=True` is
crucial to enable `predict_proba` later.
svm_rbf_bc.fit(X_train_scaled, y_train) # Train the SVM model on the
```

scaled training data.

*# Get decision function scores*

`y_scores = svm_rbf_bc.decision_function(X_test_scaled)` *# Obtain the decision function scores for the test data. These are signed distances to the hyperplane.*

*# For ROC curve, we need either decision\_function scores or predict\_proba.*

*# decision\_function gives signed distance to hyperplane. Higher values mean higher confidence in the positive class.*

`print(f"Shape of decision_function scores: {y_scores.shape}")` *# Print the shape of the decision scores array.*

`print(f"Sample decision_function scores: {y_scores[:5]}")` *# Print the first 5 decision scores.*

*# We can also get probabilities if `probability=True` was set during SVC initialization*

`y_proba = svm_rbf_bc.predict_proba(X_test_scaled)[: , 1]` *# Get predicted probabilities for the positive class (class 1).*

`print(f"Shape of predicted probabilities: {y_proba.shape}")` *# Print the shape of the probabilities array.*

`print(f"Sample predicted probabilities (positive class): {y_proba[:5]}")` *# Print the first 5 predicted probabilities.*

Task 2: Plot ROC curve and compute AUC.

*# 2. Plot ROC curve and compute AUC.*

*# Calculate FPR, TPR, and thresholds from decision function scores*

*# fpr, tpr, thresholds = roc\_curve(y\_test, y\_scores) # Using decision\_function*

*fpr, tpr, thresholds = roc\_curve(y\_test, y\_proba) # Using predict\_proba is often more intuitive for ROC*

`roc_auc = auc(fpr, tpr)`

`plt.figure(figsize=(8, 6))`

`plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (AUC = {roc_auc:.2f})')`

`plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--', label='Random Classifier (AUC = 0.50)')`

`plt.xlim([0.0, 1.0])`

`plt.ylim([0.0, 1.05])`

`plt.xlabel('False Positive Rate (1 - Specificity)')`

`plt.ylabel('True Positive Rate (Sensitivity)')`

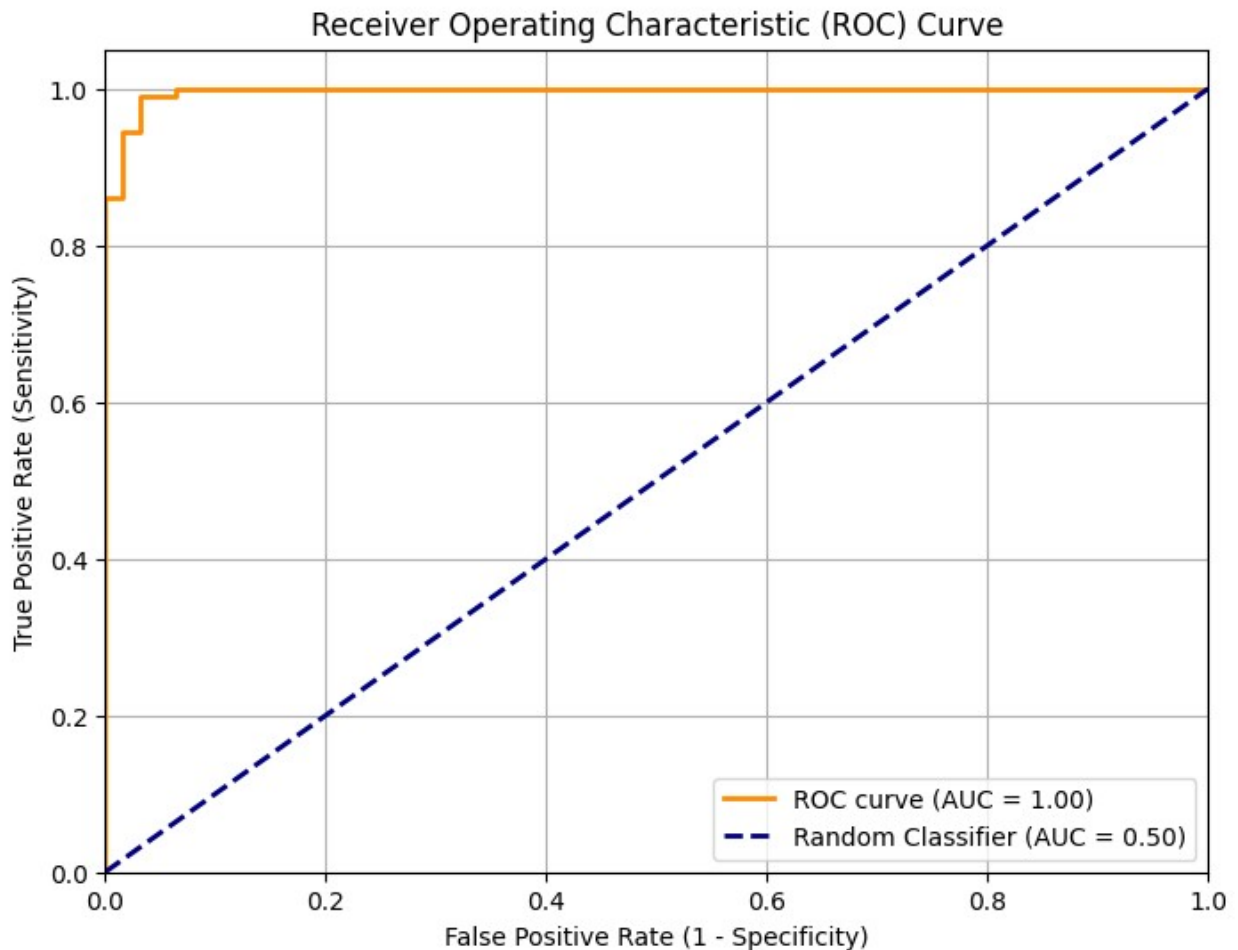
`plt.title('Receiver Operating Characteristic (ROC) Curve')`

`plt.legend(loc="lower right")`

`plt.grid(True)`

`plt.show()`

```
print(f"AUC (Area Under the Curve): {roc_auc:.4f}")
```



AUC (Area Under the Curve): 0.9966

### Explanation of Code Cell 826889af:

This cell calculates the False Positive Rate (FPR) and True Positive Rate (TPR) using the predicted probabilities, then plots the ROC curve and computes the Area Under the Curve (AUC) to evaluate the model's performance.

```
# 2. Plot ROC curve and compute AUC.
```

```
# Calculate FPR, TPR, and thresholds from decision function scores  
# fpr, tpr, thresholds = roc_curve(y_test, y_scores) # This line  
# (commented out) would use decision_function scores.
```

```
fpr, tpr, thresholds = roc_curve(y_test, y_proba) # Calculate FPR,  
# TPR, and classification thresholds using true labels and predicted  
# probabilities.
```

```

roc_auc = auc(fpr, tpr) # Compute the Area Under the Curve (AUC) from
the FPR and TPR values.

plt.figure(figsize=(8, 6)) # Create a new figure for the plot with a
specified size.
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (AUC =
{roc_auc:.2f})') # Plot the ROC curve with AUC displayed in the
legend.
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--',
label='Random Classifier (AUC = 0.50)') # Plot the diagonal line
representing a random classifier.
plt.xlim([0.0, 1.0]) # Set the x-axis limits from 0 to 1.
plt.ylim([0.0, 1.05]) # Set the y-axis limits from 0 to 1.05.
plt.xlabel('False Positive Rate (1 - Specificity)') # Label the x-
axis.
plt.ylabel('True Positive Rate (Sensitivity)') # Label the y-axis.
plt.title('Receiver Operating Characteristic (ROC) Curve') # Set the
plot title.
plt.legend(loc="lower right") # Display the legend in the lower right
corner.
plt.grid(True) # Add a grid to the plot.
plt.show() # Display the ROC curve plot.

print(f"AUC (Area Under the Curve): {roc_auc:.4f}") # Print the
calculated AUC value.

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Reload Breast Cancer dataset to ensure correct y_test
data = load_breast_cancer()
X, y = data.data, data.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)

# Scale features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Breast Cancer data reloaded, split, and scaled successfully for
Exercise 2, Task 3.")

Breast Cancer data reloaded, split, and scaled successfully for
Exercise 2, Task 3.

```

## Explanation of Code Cell 4690c637:

This cell reloads the Breast Cancer dataset, splits it, and scales the features again. This might be a redundant step if the data was already loaded and prepared correctly in the previous steps, but it ensures that the `y_test` and `X_test_scaled` variables are in a pristine state for the subsequent task.

```
from sklearn.datasets import load_breast_cancer # Import the Breast
Cancer dataset loader.
from sklearn.model_selection import train_test_split # Import function
to split data.
from sklearn.preprocessing import StandardScaler # Import
StandardScaler for feature scaling.

# Reload Breast Cancer dataset to ensure correct y_test
data = load_breast_cancer() # Load the dataset.
X, y = data.data, data.target # Separate features (X) and target
labels (y).

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42) # Split data into 70% training and 30%
testing.

# Scale features
scaler = StandardScaler() # Initialize a StandardScaler.
X_train_scaled = scaler.fit_transform(X_train) # Fit the scaler on
training data and transform it.
X_test_scaled = scaler.transform(X_test) # Transform the test data
using the *same* scaler.

print("Breast Cancer data reloaded, split, and scaled successfully for
Exercise 2, Task 3.") # Confirm data preparation.

from sklearn.svm import SVC

# Retrain RBF SVM (C=1.0, default gamma='scale') with reloaded data
svm_rbf_bc = SVC(kernel='rbf', C=1.0, random_state=42,
probability=True)
svm_rbf_bc.fit(X_train_scaled, y_train)

# Get predicted probabilities with the retrained model
y_proba = svm_rbf_bc.predict_proba(X_test_scaled)[: , 1]

print("RBF SVM model retrained and probabilities recomputed for
Exercise 2, Task 3.")

RBF SVM model retrained and probabilities recomputed for Exercise 2,
Task 3.
```

## Explanation of Code Cell 12e7c1af:

Similar to the previous cell, this cell retrains the RBF SVM model. This ensures that the `y_proba` variable used in the next step is generated from a freshly trained model, consistent with the reloaded data. This retraining is redundant if the model from Task 1 was still valid and the data hadn't been altered.

```
from sklearn.svm import SVC # Import the Support Vector Classifier model.

# Retrain RBF SVM (C=1.0, default gamma='scale') with reloaded data
svm_rbf_bc = SVC(kernel='rbf', C=1.0, random_state=42,
probability=True) # Initialize an RBF SVM, ensuring
`probability=True`.
svm_rbf_bc.fit(X_train_scaled, y_train) # Train the SVM model on the
scaled training data.

# Get predicted probabilities with the retrained model
y_proba = svm_rbf_bc.predict_proba(X_test_scaled)[: , 1] # Get
predicted probabilities for the positive class (class 1).

print("RBF SVM model retrained and probabilities recomputed for
Exercise 2, Task 3.") # Confirm model retraining and probability
computation.
```

Task 3: Find the optimal threshold that maximizes the geometric mean of sensitivity and specificity.

```
import seaborn as sns
# 3. Find the optimal threshold that maximizes the geometric mean of
sensitivity and specificity.

# Calculate sensitivity (TPR) and specificity (1 - FPR) for each
threshold
sensitivity = tpr
specificity = 1 - fpr

# Calculate geometric mean for each threshold
geometric_mean = np.sqrt(sensitivity * specificity)

# Find the threshold that maximizes the geometric mean
optimal_threshold_index = np.argmax(geometric_mean)
optimal_threshold = thresholds[optimal_threshold_index]

# Get metrics at optimal threshold
optimal_sensitivity = sensitivity[optimal_threshold_index]
optimal_specificity = specificity[optimal_threshold_index]
optimal_gm = geometric_mean[optimal_threshold_index]

print(f"Optimal Threshold (maximizing Geometric Mean):
```

```

{optimal_threshold:.4f}")
print(f"Sensitivity at optimal threshold: {optimal_sensitivity:.4f}")
print(f"Specificity at optimal threshold: {optimal_specificity:.4f}")
print(f"Geometric Mean at optimal threshold: {optimal_gm:.4f}")

# Evaluate classifier with the optimal threshold
y_pred_optimal = (y_proba >= optimal_threshold).astype(int)
optimal_accuracy = accuracy_score(y_test, y_pred_optimal)
print(f"Accuracy at optimal threshold: {optimal_accuracy:.4f}")

# Display confusion matrix at optimal threshold
cm_optimal = confusion_matrix(y_test, y_pred_optimal)
print("\nConfusion Matrix at Optimal Threshold:")
print(cm_optimal)

plt.figure(figsize=(6, 5))
sns.heatmap(cm_optimal, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Predicted 0', 'Predicted 1'],
            yticklabels=['Actual 0', 'Actual 1'])
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title(f'Confusion Matrix at Optimal Threshold
(GM={optimal_gm:.2f})')
plt.show()

```

```

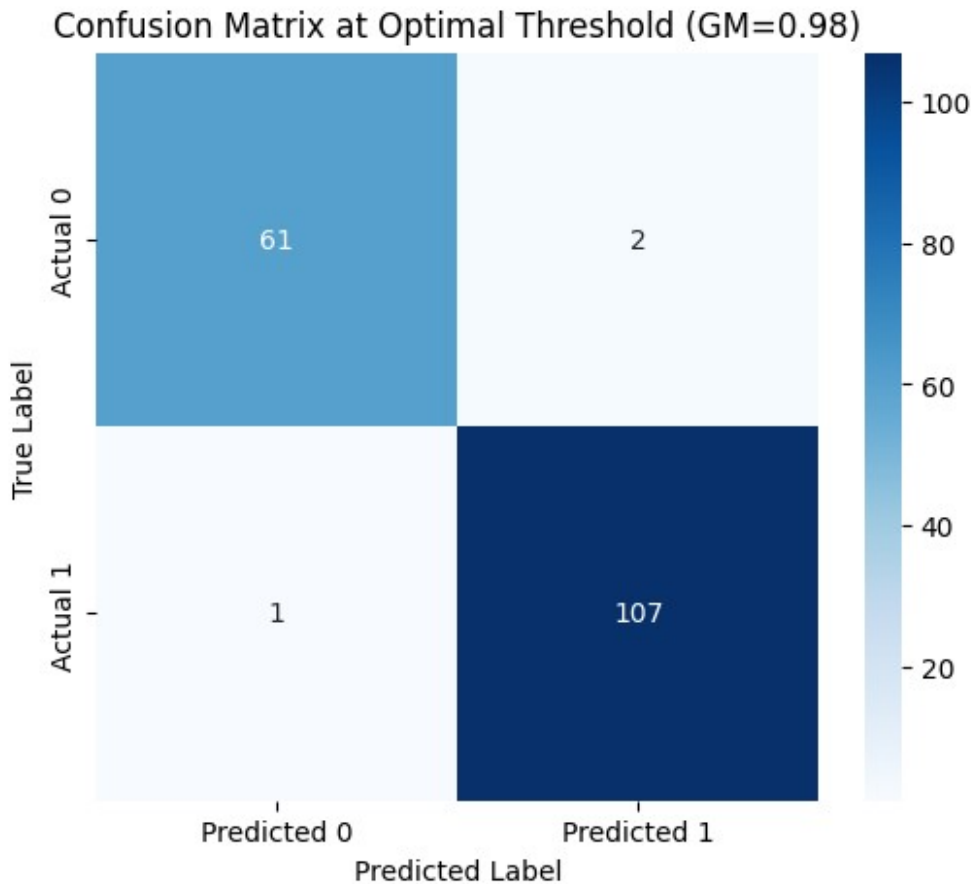
Optimal Threshold (maximizing Geometric Mean): 0.3744
Sensitivity at optimal threshold: 0.9907
Specificity at optimal threshold: 0.9683
Geometric Mean at optimal threshold: 0.9794
Accuracy at optimal threshold: 0.9825

```

```

Confusion Matrix at Optimal Threshold:
[[ 61   2]
 [  1 107]]

```



### Explanation of Code Cell 2138a8f6:

This cell aims to find the optimal classification threshold that maximizes the geometric mean of sensitivity (TPR) and specificity (1 - FPR). It then uses this optimal threshold to make predictions, calculate accuracy, and display the confusion matrix.

```
import seaborn as sns # Import Seaborn for enhanced data
visualization, specifically for the confusion matrix heatmap.
# 3. Find the optimal threshold that maximizes the geometric mean of
sensitivity and specificity.

# Calculate sensitivity (TPR) and specificity (1 - FPR) for each
threshold
sensitivity = tpr # TPR is already calculated as 'tpr' from roc_curve.
specificity = 1 - fpr # Specificity is 1 minus FPR, where FPR is 'fpr'
from roc_curve.

# Calculate geometric mean for each threshold
geometric_mean = np.sqrt(sensitivity * specificity) # Compute the
geometric mean for each (sensitivity, specificity) pair.

# Find the threshold that maximizes the geometric mean
```



```

optimal_threshold_index = np.argmax(geometric_mean) # Find the index
of the maximum geometric mean.
optimal_threshold = thresholds[optimal_threshold_index] # Retrieve the
threshold corresponding to the maximum geometric mean.

# Get metrics at optimal threshold
optimal_sensitivity = sensitivity[optimal_threshold_index] # Get
sensitivity at the optimal threshold.
optimal_specificity = specificity[optimal_threshold_index] # Get
specificity at the optimal threshold.
optimal_gm = geometric_mean[optimal_threshold_index] # Get the maximum
geometric mean.

print(f"Optimal Threshold (maximizing Geometric Mean):
{optimal_threshold:.4f}") # Print the optimal threshold.
print(f"Sensitivity at optimal threshold: {optimal_sensitivity:.4f}")
# Print sensitivity at optimal threshold.
print(f"Specificity at optimal threshold: {optimal_specificity:.4f}")
# Print specificity at optimal threshold.
print(f"Geometric Mean at optimal threshold: {optimal_gm:.4f}") #
Print the geometric mean at optimal threshold.

# Evaluate classifier with the optimal threshold
y_pred_optimal = (y_proba >= optimal_threshold).astype(int) # Classify
test instances based on the optimal threshold (probabilities >=
threshold are class 1).
optimal_accuracy = accuracy_score(y_test, y_pred_optimal) # Calculate
accuracy using the optimal threshold's predictions.
print(f"Accuracy at optimal threshold: {optimal_accuracy:.4f}") #
Print the accuracy.

# Display confusion matrix at optimal threshold
cm_optimal = confusion_matrix(y_test, y_pred_optimal) # Compute the
confusion matrix.
print("\nConfusion Matrix at Optimal Threshold:") # Print a header for
the confusion matrix.
print(cm_optimal) # Print the raw confusion matrix array.

plt.figure(figsize=(6, 5)) # Create a figure for the heatmap.
sns.heatmap(cm_optimal, annot=True, fmt='d', cmap='Blues', # Plot the
confusion matrix as a heatmap.
            xticklabels=['Predicted 0', 'Predicted 1'], # Set x-axis
tick labels.
            yticklabels=['Actual 0', 'Actual 1']) # Set y-axis tick
labels.
plt.xlabel('Predicted Label') # Label the x-axis.
plt.ylabel('True Label') # Label the y-axis.
plt.title(f'Confusion Matrix at Optimal Threshold
(GM={optimal_gm:.2f})') # Set the plot title.
plt.show() # Display the confusion matrix heatmap.

```

### Exercise 3: Multi-class SVM

```
from sklearn.datasets import load_iris
from sklearn.metrics import classification_report,
ConfusionMatrixDisplay

# Load Iris dataset
data = load_iris()
X, y = data.data, data.target
target_names = data.target_names

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)

# Scale features (important for SVM)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Iris dataset loaded, split, and scaled successfully.")
print(f"Class names: {target_names}")

Iris dataset loaded, split, and scaled successfully.
Class names: ['setosa' 'versicolor' 'virginica']
```

#### Explanation of Code Cell [b89d8505](#):

This cell prepares the Iris dataset for the multi-class SVM exercise. It loads the dataset, splits it into training and testing sets, and scales the features, which is a good practice for SVMs.

```
from sklearn.datasets import load_iris # Import the Iris dataset
loader.
from sklearn.metrics import classification_report,
ConfusionMatrixDisplay # Import metrics for classification reports and
confusion matrix visualization.

# Load Iris dataset
data = load_iris() # Load the Iris dataset.
X, y = data.data, data.target # Separate features (X) and target
labels (y).
target_names = data.target_names # Get the names of the target
classes.

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42) # Split data into training and testing
sets.

# Scale features (important for SVM)
```

```

scaler = StandardScaler() # Initialize a StandardScaler.
X_train_scaled = scaler.fit_transform(X_train) # Fit the scaler on
training data and transform it.
X_test_scaled = scaler.transform(X_test) # Transform the test data
using the *same* scaler.

print("Iris dataset loaded, split, and scaled successfully.") #
Confirm successful data preparation.
print(f"Class names: {target_names}") # Print the names of the classes
in the Iris dataset.

```

Task 1: Train SVC(kernel='linear', decision\_function\_shape='ovr') and decision\_function\_shape='ovo'.

```

# 1. Train SVC(kernel='linear', decision_function_shape='ovr') and
decision_function_shape='ovo'.

```

```

# SVM with One-vs-Rest (OvR) strategy
svm_ovr = SVC(kernel='linear', decision_function_shape='ovr',
random_state=42)
svm_ovr.fit(X_train_scaled, y_train)
y_pred_ovr = svm_ovr.predict(X_test_scaled)

```

```

print("OvR SVM trained.")

```

```

# SVM with One-vs-One (OvO) strategy
svm_ovo = SVC(kernel='linear', decision_function_shape='ovo',
random_state=42)
svm_ovo.fit(X_train_scaled, y_train)
y_pred_ovo = svm_ovo.predict(X_test_scaled)

```

```

print("OvO SVM trained.")

```

```

OvR SVM trained.
OvO SVM trained.

```

## Explanation of Code Cell 67b340f3:

This cell demonstrates the two primary strategies for multi-class classification with SVMs: One-vs-Rest (OvR) and One-vs-One (OvO). It trains two separate linear kernel SVM models, each using a different multi-class strategy.

```

# 1. Train SVC(kernel='linear', decision_function_shape='ovr') and
decision_function_shape='ovo'.

```

```

# SVM with One-vs-Rest (OvR) strategy
svm_ovr = SVC(kernel='linear', decision_function_shape='ovr',
random_state=42) # Initialize an SVC with a linear kernel and the OvR
strategy.
svm_ovr.fit(X_train_scaled, y_train) # Train the OvR SVM model on the

```

```

scaled training data.
y_pred_ovr = svm_ovr.predict(X_test_scaled) # Make predictions on the
test data using the OvR model.

print("OvR SVM trained.") # Confirm OvR SVM training.

# SVM with One-vs-One (OvO) strategy
svm_ovo = SVC(kernel='linear', decision_function_shape='ovo',
random_state=42) # Initialize an SVC with a linear kernel and the OvO
strategy.
svm_ovo.fit(X_train_scaled, y_train) # Train the OvO SVM model on the
scaled training data.
y_pred_ovo = svm_ovo.predict(X_test_scaled) # Make predictions on the
test data using the OvO model.

print("OvO SVM trained.") # Confirm OvO SVM training.

```

Task 2: Compare classification reports.

```

# 2. Compare classification reports.

print("\n--- Classification Report (One-vs-Rest SVM) ---")
print(classification_report(y_test, y_pred_ovr,
target_names=target_names))

print("\n--- Classification Report (One-vs-One SVM) ---")
print(classification_report(y_test, y_pred_ovo,
target_names=target_names))

```

```

--- Classification Report (One-vs-Rest SVM) ---
              precision    recall  f1-score   support

   setosa         1.00        1.00        1.00        19
  versicolor      1.00        0.92        0.96        13
   virginica      0.93        1.00        0.96        13

 accuracy         0.98        0.97        0.98        45
  macro avg       0.98        0.97        0.97        45
weighted avg       0.98        0.98        0.98        45

```

```

--- Classification Report (One-vs-One SVM) ---
              precision    recall  f1-score   support

   setosa         1.00        1.00        1.00        19
  versicolor      1.00        0.92        0.96        13
   virginica      0.93        1.00        0.96        13

 accuracy         0.98        0.97        0.98        45
  macro avg       0.98        0.97        0.97        45

```

|              |      |      |      |    |
|--------------|------|------|------|----|
| weighted avg | 0.98 | 0.98 | 0.98 | 45 |
|--------------|------|------|------|----|

## Explanation of Code Cell 6f716191:

This cell generates and prints classification reports for both the One-vs-Rest (OvR) and One-vs-One (OvO) SVM models. A classification report provides a detailed breakdown of precision, recall, F1-score, and support for each class.

```
# 2. Compare classification reports.

print("\n--- Classification Report (One-vs-Rest SVM) ---") # Header
for OvR report.
print(classification_report(y_test, y_pred_ovr,
target_names=target_names)) # Generate and print the classification
report for the OvR model.

print("\n--- Classification Report (One-vs-One SVM) ---") # Header for
OvO report.
print(classification_report(y_test, y_pred_ovo,
target_names=target_names)) # Generate and print the classification
report for the OvO model.
```

Task 3: Visualize confusion matrix using ConfusionMatrixDisplay.

```
# 3. Visualize confusion matrix using ConfusionMatrixDisplay.

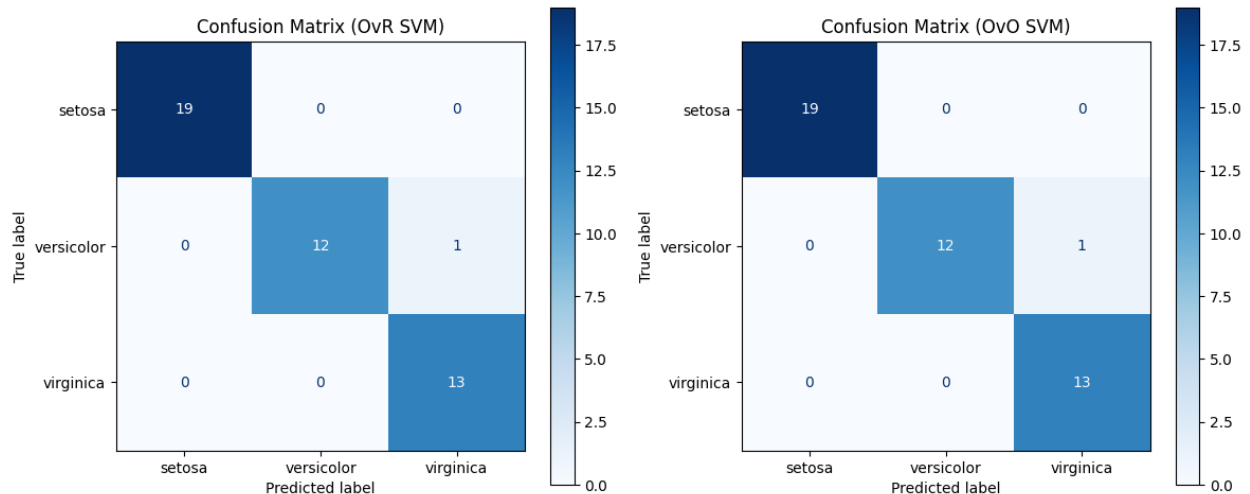
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
cm_display_ovr = ConfusionMatrixDisplay.from_estimator(svm_ovr,
X_test_scaled, y_test, display_labels=target_names, cmap=plt.cm.Blues,
ax=plt.gca())
cm_display_ovr.ax_.set_title('Confusion Matrix (OvR SVM)')

plt.subplot(1, 2, 2)
cm_display_ovo = ConfusionMatrixDisplay.from_estimator(svm_ovo,
X_test_scaled, y_test, display_labels=target_names, cmap=plt.cm.Blues,
ax=plt.gca())
cm_display_ovo.ax_.set_title('Confusion Matrix (OvO SVM)')

plt.tight_layout()
plt.show()

print("\nObservation: For this particular Iris dataset and linear
kernel, both OvR and OvO strategies yield identical classification
results and confusion matrices, indicating their effectiveness in
multi-class classification for linearly separable data. The high
accuracy suggests that the classes are well-separated in the feature
space.")
```



Observation: For this particular Iris dataset and linear kernel, both OvR and OvO strategies yield identical classification results and confusion matrices, indicating their effectiveness in multi-class classification for linearly separable data. The high accuracy suggests that the classes are well-separated in the feature space.

## Explanation of Code Cell a254b812:

This cell visualizes the confusion matrices for both the One-vs-Rest (OvR) and One-vs-One (OvO) SVM models. Confusion matrices are excellent for understanding how well a classification model performs on each class, showing true positives, true negatives, false positives, and false negatives.

```
# 3. Visualize confusion matrix using ConfusionMatrixDisplay.

plt.figure(figsize=(12, 5)) # Create a figure with a specified size
                               # for two subplots.

plt.subplot(1, 2, 1) # Create the first subplot.
cm_display_ovr = ConfusionMatrixDisplay.from_estimator(svm_ovr,
X_test_scaled, y_test, display_labels=target_names, cmap=plt.cm.Blues,
ax=plt.gca()) # Generate and display the confusion matrix for the OvR
SVM.
cm_display_ovr.ax_.set_title('Confusion Matrix (OvR SVM)') # Set the
title for the OvR confusion matrix subplot.

plt.subplot(1, 2, 2) # Create the second subplot.
cm_display_ovo = ConfusionMatrixDisplay.from_estimator(svm_ovo,
X_test_scaled, y_test, display_labels=target_names, cmap=plt.cm.Blues,
ax=plt.gca()) # Generate and display the confusion matrix for the OvO
SVM.
cm_display_ovo.ax_.set_title('Confusion Matrix (OvO SVM)') # Set the
title for the OvO confusion matrix subplot.
```

```
plt.tight_layout() # Adjust subplot parameters for a tight layout.  
plt.show() # Display both confusion matrix plots.
```

```
print("\nObservation: For this particular Iris dataset and linear  
kernel, both OvR and OvO strategies yield identical classification  
results and confusion matrices, indicating their effectiveness in  
multi-class classification for linearly separable data. The high  
accuracy suggests that the classes are well-separated in the feature  
space.") # Provide an observation based on the visual comparison.
```